

Portfólio Pessoal

Fernando Melo

Site pessoal de vitrine técnica — Next.js + GitHub API
do PRD ao deploy em produção

AUTOR	Fernando José da Silva Melo
REPOSITÓRIO	github.com/fernandojsmelo/portfolio-pessoal
PRODUÇÃO	portfolio-pessoal-six-sigma.vercel.app
PERÍODO DE EXECUÇÃO	16–18 de setembro de 2026
STATUS ATUAL	No ar — 8/8 fases do plano concluídas (1 pendência operacional)

Next.js 16

TypeScript

React 19

Tailwind CSS v4

Vercel

GitHub API

§00 — SUMÁRIO EXECUTIVO

Sumário Executivo

Este relatório documenta o ciclo completo de construção do site pessoal de portfólio de Fernando Melo: da definição de requisitos (PRD) ao plano de implementação faseado, passando pelo design system próprio, pela integração com a API do GitHub e pelo deploy em produção na Vercel. O projeto foi conduzido de forma assistida por IA (Claude Code), com documentação versionada em Git a cada etapa.

Fases do plano concluídas	8 de 8 (100%) — 1 pendência operacional isolada
Projetos publicados no site	5 projetos curados manualmente, com case técnico real
Commits registrados	14 commits, do scaffold inicial ao ajuste final de contato
Stack principal	Next.js 16 (App Router) · TypeScript · React 19 · Tailwind CSS v4
Renderização	SSG com ISR (revalidação a cada 24h) — sem backend, sem banco de dados
Ambiente de produção	Vercel — portfolio-pessoal-six-sigma.vercel.app
Pendência conhecida	Deploy automático via Git ainda não conectado (deploy manual funcional)

§01 — VISÃO GERAL E OBJETIVO

Visão Geral e Objetivo

O site foi concebido como ferramenta de marca pessoal, com o objetivo de divulgar os projetos tecnicamente mais significativos do GitHub de Fernando (github.com/fernandojsmelo) para um público amplo — recrutadores, potenciais clientes e comunidade técnica — combinando uma vitrine curada de projetos com uma apresentação pessoal e profissional.

O escopo do MVP, definido em **PRD.md**, foi mantido deliberadamente enxuto: uma home de página única (hero, projetos em destaque, sobre, contato) e uma página de detalhe por projeto (**/projetos/[slug]**), com 6 a 10 projetos curados manualmente. Ficaram fora de escopo, por decisão explícita: blog/CMS, internacionalização, busca avançada, formulário de contato com backend, autenticação e testes E2E automatizados.

§02 — ARQUITETURA E STACK TÉCNICA

Arquitetura e Stack Técnica

O site é inteiramente estático: sem backend próprio e sem banco de dados. A renderização é feita via SSG (Static Site Generation) em build-time, com ISR (Incremental Static Regeneration) revalidando os dados a cada 24 horas, o suficiente para manter os metadados do GitHub razoavelmente atualizados sem exigir redeploy manual.

Camada	Escolha técnica
Framework	Next.js 16.3.5 (App Router)
Linguagem	TypeScript
UI	React 19.2.8 / React DOM 19.2.8
Estilização	Tailwind CSS v4 (CSS-first, via @theme)

Camada	Escolha técnica
Conteúdo	gray-matter (front-matter) + remark / remark-html (Markdown → HTML)
Testes	Vitest
Lint	ESLint (eslint-config-next)
Hospedagem	Vercel, com deploy automático planejado a cada push na branch principal

Nenhuma chamada à API do GitHub acontece no navegador do visitante — todos os dados dinâmicos já chegam pré-renderizados no HTML servido.

§03 — MODELO DE DADOS HÍBRIDO

Modelo de Dados Híbrido

Cada projeto vive como um arquivo Markdown com front-matter em `content/projects/{slug}.md`, combinando curadoria manual com enriquecimento automático:

Origem	Conteúdo
Curado manualmente	slug, title, repo, tagline, techStack, featured, order, screenshot?, liveUrl?, corpo Markdown (case técnico)
Buscado da API do GitHub	stars, primaryLanguage, lastUpdated, openIssues — via <code>api.github.com/repos/{repo}</code> , autenticado com <code>GITHUB_TOKEN</code>
Combinação	<code>getAllProjects()</code> une front-matter + resposta da API, ordenado por <code>order</code> , alimentando tanto o card da home quanto a página de detalhe

Degradação graciosa: se a API do GitHub falhar, estiver com rate limit atingido ou o repositório referenciado não existir mais, o build não quebra — a entrada usa apenas os dados curados (sem stars/linguagem) e um aviso é logado no processo de build. O rate limit é elevado de 60 para 5.000 requisições/hora via Personal Access Token (PAT) do GitHub nas variáveis de ambiente de build.

§04 — DIREÇÃO DE DESIGN

Direção de Design

A home foi tratada como um dossiê/datasheet técnico: numeração de seção real (§01, §02...), cards de projeto como fichas técnicas com índice de spec, e ausência total de cantos arredondados — hairlines fazem a separação visual, não sombras. A direção foi validada num mockup aprovado antes da implementação.

Paleta de cores

Cor	Token	Hex	Uso
	ink	#15130f	Fundo
	surface	#201d18	Cards
	line	#3a352c	Hairlines
	paper	#f3ede1	Texto principal
	muted	#a69c8a	Texto secundário
	signal	#ff5c1a	Acento único

Tipografia

Fonte	Papel
Fraunces	Display / headlines
IBM Plex Sans	Corpo de texto
IBM Plex Mono	Badges, metadados, numeração de seção

Tema único dark, sem alternância light/dark (decisão de escopo, não pendência). Contraste verificado contra WCAG AA — todos os pares de cor de texto acima de 4.5:1.

§05 — LINHA DO TEMPO DE EXECUÇÃO

Linha do Tempo de Execução

O plano de implementação (PLANO_IMPLEMENTACAO.md) organizou o trabalho em 8 fases sequenciais (0 a 7), cada uma dependente da anterior estar funcional. Todas as 14 entregas git abaixo ocorreram entre 16 e 18 de setembro de 2026.

Fase	Entrega	Status	Notas
Fase 0	Scaffold do projeto	Concluída	create-next-app, git init, .env.local.example, .gitignore
Fase 1	Design tokens e fontes	Concluída	Google Fonts via next/font, tokens de cor em CSS variables, Tailwind v4 CSS-first
Fase 2	Modelo de dados dos projetos	Concluída	Tipo Project, parser front-matter, getGithubMetadata(), degradação graciosa, testes unitários
Fase 3	Componentes de UI	Concluída	Header, Footer, TechBadge, ProjectCard, SectionEyebrow
Fase 4	Página Home	Concluída	Hero, §01 Sobre, §02 Projetos, §03 Contato, rail lateral, responsividade ~400px
Fase 5	Página de detalhe do projeto	Concluída	generateStaticParams, layout completo, ISR 24h, 404 padrão
Fase 6	Conteúdo real	Concluída	5 projetos curados com case técnico real (sem screenshots ainda)
Fase 7	QA final e deploy	Concluída (1 pendência)	Build limpo, checklist visual, contraste AA, deploy na Vercel; falta conectar Git para deploy automático

Histórico de commits

Data	Commit	Mensagem
16/09	3692e8d	docs: add PRD, CLAUDE.md e plano de implementação inicial
16/09	cffc69f	chore: add Claude Code skills do projeto
16/09	00297cd	feat: scaffold do projeto Next.js (Fase 0)
16/09	86a3585	feat: tokens de design e fontes (Fase 1)
16/09	813c447	feat: modelo de dados dos projetos (Fase 2)
16/09	ee7abe1	feat: componentes de UI do design system (Fase 3)
16/09	d7ca428	feat: página home (Fase 4)
16/09	f3df584	feat: página de detalhe do projeto (Fase 5)
16/09	c20b596	feat: conteúdo real dos projetos curados (Fase 6)
16/09	a8cbb92	chore: limpeza final de QA antes do deploy (Fase 7)
16/09	f32292d	docs: marca fases 0-7 do plano como concluídas
16/09	5735067	fix: corrige link do LinkedIn quebrado em site-config
16/09	ed4d759	content: atualiza hero, sobre e experiência/formação reais
18/09	8c42f82	fix: exibe e-mail e usuário do LinkedIn nos links de contato

§06 — PROJETOS PUBLICADOS

Projetos Publicados

Cinco projetos foram selecionados manualmente entre os cerca de 40 repositórios públicos do GitHub de Fernando. A curadoria priorizou profundidade técnica real sobre volume — a maioria dos repositórios restantes era material de curso/tutorial, e a decisão foi de curadoria honesta em vez de forçar exercícios como "projetos significativos".

#	Projeto	Stack	Tagline
1	The Game Awards Explorer	TypeScript, Next.js, SQLite	Explorador do histórico de indicações e vencedores do The Game Awards, com filtros combinados e gráficos agregados.
2	Controle de Gastos Pessoais API	Python, FastAPI, SQLite	API construída com Spec Driven Development — specs, testes e documentação como parte do processo.
3	Dashboard Oscar	Python, Streamlit	Dashboard interativo com o histórico de indicações e vencedores do Oscar entre 1927 e 2024.
4	Aniversariantes do Mês	Node.js, Systemd	Automação que identifica aniversariantes do mês e envia lembretes por SMS, agendada via systemd.
5	Chess System	Java	Motor de xadrez em Java, com as regras completas do jogo numa arquitetura orientada a objetos em camadas.

Cada entrada expõe metadados ao vivo do GitHub (stars, linguagem principal, última atualização, issues abertas) na página de detalhe, além do case técnico (problema + decisões técnicas) escrito manualmente. Nenhum dos cinco projetos tinha screenshot pronto para reaproveitar — item deixado como melhoria futura, por projeto.

§07 — QUALIDADE E TESTES

Qualidade e Testes

Frente	Cobertura
Testes unitários	Cobrem a função de combinação de dados (front-matter + resposta da API), incluindo caso de sucesso e caso de falha da API, via Vitest
Smoke test de build	npm run build completa sem erros com dados reais e GITHUB_TOKEN válido
Checklist visual manual	Mobile (~400px) e desktop, nas duas páginas (home e detalhe)
Contraste (WCAG AA)	Todos os pares de cor de texto do tema dark verificados acima de 4.5:1
Testes E2E	Fora de escopo do MVP por decisão do PRD — pode ser reconsiderado se o site ganhar lógica própria

§08 — DEPLOY E INFRAESTRUTURA

Deploy e Infraestrutura

Item	Detalhe
Plataforma	Vercel — projeto fernando-melo1/portfolio-pessoal
URL de produção	https://portfolio-pessoal-six-sigma.vercel.app (domínio padrão da Vercel, por decisão do usuário)
Variável de build	GITHUB_TOKEN — fine-grained PAT, somente leitura pública, configurado nas env vars de produção
Revalidação	ISR a cada 86400s (24h) por rota de projeto
Forma de deploy atual	Manual, via vercel --prod

§09 — PENDÊNCIAS E PRÓXIMOS PASSOS

Pendências e Próximos Passos

Pendência aberta: conectar o repositório GitHub ao projeto na Vercel para habilitar deploy automático a cada push na branch principal. A tentativa via **vercel git connect** falhou — é necessário autorizar o GitHub App da Vercel manualmente pelo dashboard (Project → Settings → Git → Connect Git Repository). Até essa conexão ser feita, o deploy permanece manual e funcional.

Melhorias futuras identificadas, sem compromisso de prazo:

- Adicionar screenshots aos cards de projeto (public/images/projetos/), quando houver material disponível por projeto.
- Avaliar domínio próprio, hoje o site usa o domínio padrão da Vercel.
- Reconsiderar testes E2E automatizados caso o site ganhe funcionalidades com lógica própria (ex.: formulário de contato com backend).

§10 — CONCLUSÃO

Conclusão

O projeto atingiu os critérios de sucesso definidos no PRD: site no ar na Vercel, cinco projetos curados publicados com página de detalhe funcional, dados de stars/linguagem/última atualização puxados corretamente da API do GitHub via ISR, e experiência responsiva e legível em mobile. A única pendência remanescente é operacional (conexão de deploy automático) e não bloqueia o uso do site em produção.